

FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO
CAMPUS LIBERDADE
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

GABRIEL CARVALHO MOTA
GUILHERME DE LIMA SIQUEIRA
RODRIGO LUIZ MENEZES DOS REIS
VITORIA LETICIA MACIEL DA SILVA

PICMONEY
ANÁLISE TÉCNICA DE ENGENHARIA DE SOFTWARE E DIAGRAMAS
UML

São Paulo, 2025

Sumário

OBJETIVO	3
CONCEITOS DE ENGENHARIA DE SOFTWARE	3
PRINCÍPIOS SOLID	3
PRÁTICAS DE DEVOPS E ORGANIZAÇÃO DO PROJETO	4
MANUTENÇÃO E EVOLUÇÃO	4
PADRÕES DE PROJETO UTILIZADOS	4
ARQUITETURA DO SISTEMA	5
DIAGRAMAS UML	6
Diagrama de Classes	6
Diagrama de Sequência	7
RESULTADOS E EXEMPLOS NO CÓDIGO	8
Exemplo 1 — Template Method + Strategy Pattern.....	8
Exemplo 2 — Facade Pattern + Cache Strategy.....	9
CONSIDERAÇÕES FINAIS	9

OBJETIVO

O objetivo deste documento é demonstrar como os conceitos de Engenharia de Software foram aplicados no desenvolvimento do PicMoney. A análise busca evidenciar como a organização do sistema, a aplicação de boas práticas e o uso de princípios estruturais contribuíram para um projeto funcional, escalável e de fácil manutenção.

Entre os principais objetivos, destacam-se:

- Aplicar conceitos de design de software e arquitetura;
- Incorporar princípios SOLID e padrões de projeto;
- Representar graficamente o sistema por meio de diagramas UML;
- Avaliar os resultados obtidos com essas práticas no desenvolvimento do PicMoney.

CONCEITOS DE ENGENHARIA DE SOFTWARE

Durante o desenvolvimento, foram aplicados fundamentos de Engenharia de Software com foco na organização modular e na clareza estrutural. O backend foi dividido em módulos específicos — autenticação, dashboards e gerenciamento de dados — enquanto o frontend, desenvolvido em Streamlit, concentrou a interface visual e as interações com o usuário.

A aplicação do baixo acoplamento e da alta coesão permitiu que as partes do sistema funcionassem de forma independente, reduzindo o impacto de alterações e facilitando futuras manutenções.

PRINCÍPIOS SOLID

Os princípios SOLID foram utilizados como base para garantir um código mais limpo, coeso e sustentável. Entre as práticas adotadas:

- **Responsabilidade Única:** cada classe e função foi desenvolvida para executar apenas uma tarefa;
- **Aberto/Fechado:** novas funcionalidades podem ser adicionadas sem necessidade de modificar o código já existente;
- **Inversão de Dependência:** o frontend se comunica com o backend por meio de uma interface (BackendAPI), reduzindo dependências diretas;
- **Substituição de Liskov e Segregação de Interface:** o código foi estruturado para ser reutilizável e compatível com possíveis extensões futuras.

Esses princípios contribuíram para uma arquitetura mais organizada e flexível.

PRÁTICAS DE DEVOPS E ORGANIZAÇÃO DO PROJETO

Foram aplicadas práticas de **DevOps** e organização para garantir estabilidade e portabilidade do sistema. As configurações foram centralizadas em arquivos como `.env` e `config.py`, facilitando a adaptação a diferentes ambientes.

As dependências foram gerenciadas por meio do arquivo `requirements.txt`, assegurando reprodutibilidade. No Streamlit, o tratamento de erros e mensagens de status oferece uma experiência mais fluida ao usuário.

Essas práticas favorecem a manutenção e a implantação do sistema em novos ambientes.

MANUTENÇÃO E EVOLUÇÃO

A arquitetura do PicMoney foi planejada para **facilitar sua evolução** ao longo do tempo. A adição de novos dashboards, perfis de usuário e funcionalidades pode ser realizada sem comprometer a estrutura já existente.

Funções como `carregar_dados_mysql()` e `processar_pergunta_com_sql()` foram desenvolvidas de forma modular e reutilizável, permitindo testes e ajustes de forma isolada. Além disso, comentários e docstrings foram utilizados para melhorar a legibilidade e a compreensão do código.

PADRÕES DE PROJETO UTILIZADOS

Durante a implementação, alguns **padrões de projeto** foram identificados e aplicados:

- **Singleton:** garante uma única instância de sessão ativa no Streamlit;
- **Facade:** simplifica a comunicação entre o frontend e o backend através da classe `BackendAPI`;
- **Decorator:** utilizado para controle de login e permissões de acesso;
- **Template Method:** aplicado em funções estruturadas, como `processar_pergunta_com_sql()`, que seguem um fluxo fixo com etapas personalizáveis.

Esses padrões contribuíram para um código mais padronizado, limpo e fácil de compreender.

ARQUITETURA DO SISTEMA

O PicMoney segue uma **arquitetura em camadas**, organizada da seguinte forma:

- **Interface (Frontend):** desenvolvida em Streamlit, responsável pela interação com o usuário;
- **Lógica de Aplicação:** concentra o processamento e as regras de negócio;
- **Domínio (Modelos e Regras):** classes e estruturas que representam os dados do sistema;
- **Acesso a Dados:** responsável pela comunicação com o banco de dados MySQL.

O sistema utiliza o modelo **cliente-servidor**, onde o frontend se comunica com o backend por meio de uma **API REST**, o que garante independência entre as camadas e facilidade de manutenção.

DIAGRAMAS UML

Para representar a estrutura e o funcionamento do sistema, foram elaborados dois principais diagramas UML.

Diagrama de Classes

Apresenta as entidades centrais, como Usuario, Perfil e Transacao, e seus relacionamentos

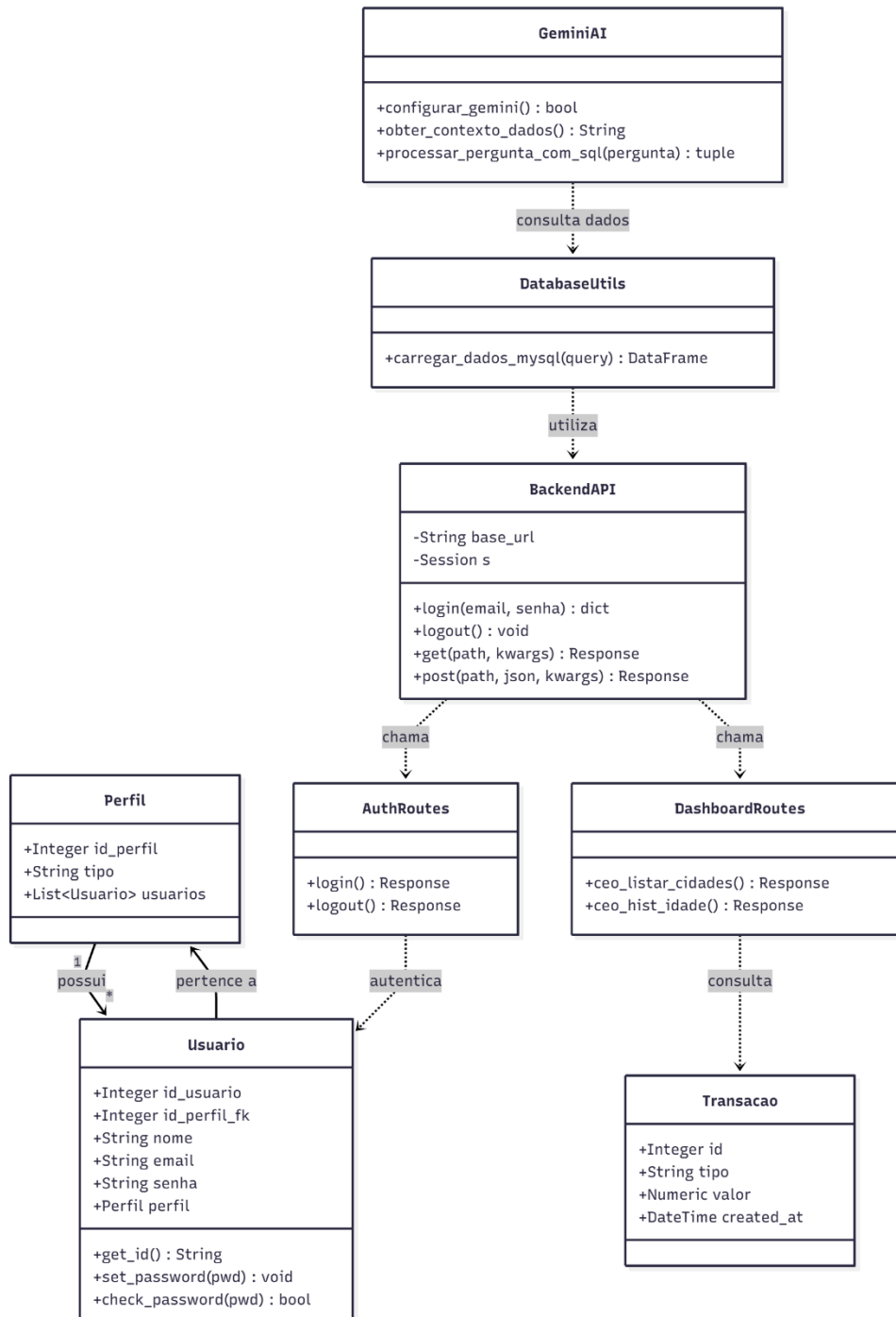
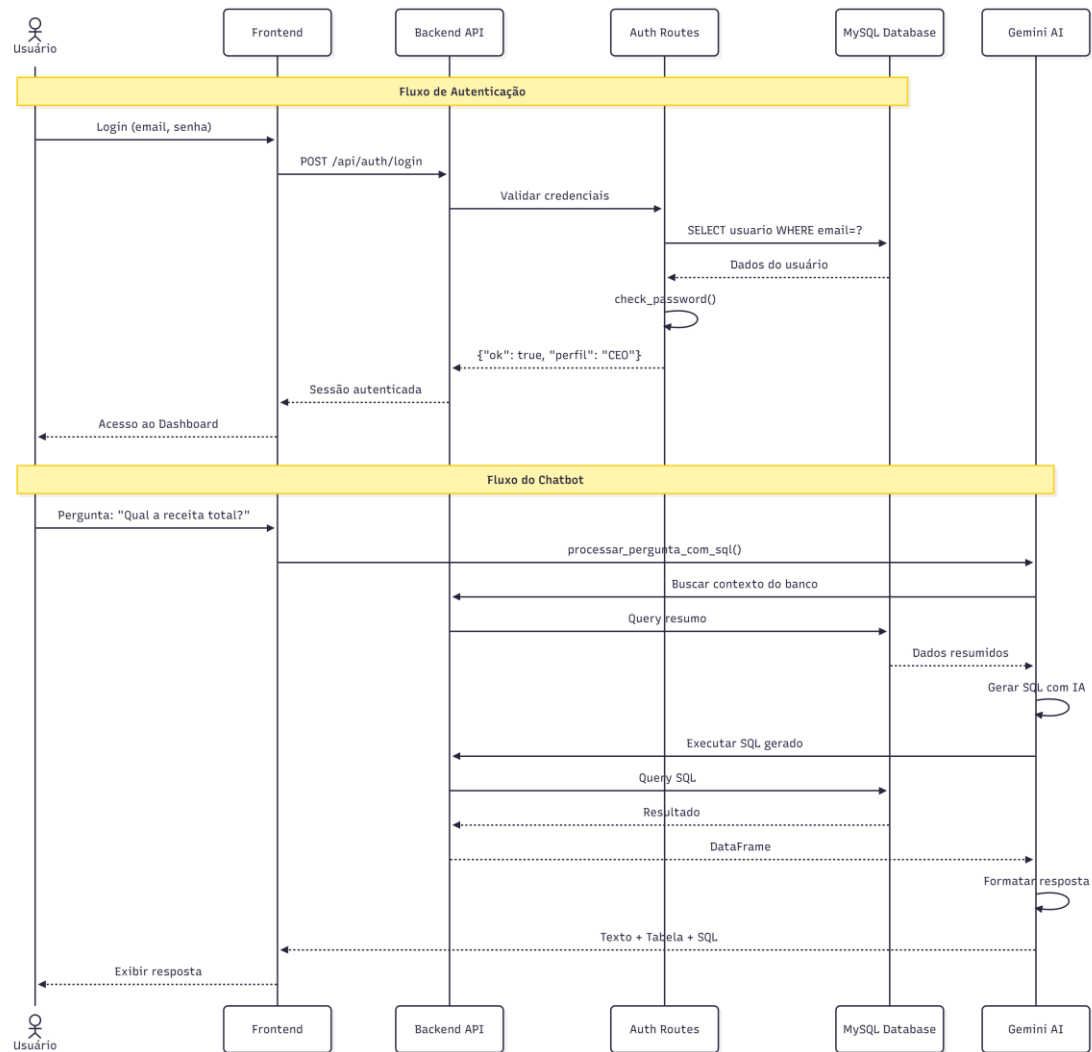


Diagrama de Sequência

Descreve o fluxo de interação entre o usuário, o frontend, o backend e a IA Gemini, ilustrando o processo de login e comunicação com o chatbot.



RESULTADOS E EXEMPLOS NO CÓDIGO

Exemplo 1 — Template Method + Strategy Pattern

Arquivo: chatbot.py

```
def processar_pergunta_com_sql(pergunta):
    """Usa o Gemini para gerar uma query SQL e executá-la no banco"""
    try:
        contexto = obter_contexto_dados()
        model = genai.GenerativeModel('gemini-2.5-flash')

        # ETAPA 1: Identificar valores/termos que precisam ser verificados no banco
        response_extracao = model.generate_content(prompt_extracao)
        query_verificacao = response_extracao.text.strip()

        # ETAPA 2: Gerar query final com contexto completo
        response_sql = model.generate_content(prompt_sql)
        sql_query = response_sql.text.strip()

        df_resultado = carregar_dados_mysql(sql_query)

        # ETAPA 3: Formatar resposta em linguagem natural
        response_final = model.generate_content(prompt_resposta)
        return response_final.text, df_resultado, sql_query

    except Exception as e:
        return f"Erro ao processar pergunta: {e}", None, None
```

Justificativa:

Esse trecho exemplifica o Template Method Pattern, com um fluxo fixo dividido em três etapas (extração, geração e formatação).

Também aplica o Strategy Pattern, ao utilizar prompts diferentes em cada fase do processamento. A função mostra modularidade e facilita a manutenção e os testes isolados de cada parte.

Exemplo 2 — Facade Pattern + Cache Strategy

Arquivo: database.py

```
@st.cache_data(ttl=600)
def carregar_dados_mysql(query: str) -> pd.DataFrame:
    """
    Encaminha a query para o backend Flask (/api/query),
    usando a sessão autenticada (cookies do Flask-Login).
    """
    try:
        api = BackendAPI()
        resp = api.post("/api/query", json={"sql": query})
        data = resp.json() if resp.ok else {}
        if not data or not data.get("ok"):
            msg = data.get("error", f"http_{resp.status_code}")
            st.error(f"Falha ao consultar o backend: {msg}")
            return pd.DataFrame()
        return pd.DataFrame(data.get("rows", []))
    except Exception as e:
        st.error(f"Erro ao consultar o backend: {e}")
        return pd.DataFrame()
```

Justificativa:

Esse trecho representa o *Facade Pattern*, ao esconder a complexidade de comunicação HTTP e autenticação em uma interface simples. Aplica também uma *Cache Strategy* com o uso do decorator `@st.cache_data`, melhorando a performance, e reforça a **Arquitetura em Camadas**, ao manter o acesso a dados separado da interface de usuário.

CONSIDERAÇÕES FINAIS

O desenvolvimento da Dashboard do PicMoney permitiu a aplicação prática de diversos conceitos da Engenharia de Software, resultando em um sistema organizado, escalável e funcional.

Os diagramas UML auxiliaram na visualização da estrutura e do fluxo de dados, enquanto os princípios SOLID e padrões de projeto garantiram clareza e facilidade de manutenção.

Como resultado, o projeto apresenta uma base sólida para futuras evoluções, refletindo o aprendizado e a aplicação efetiva das boas práticas estudadas ao longo da disciplina.